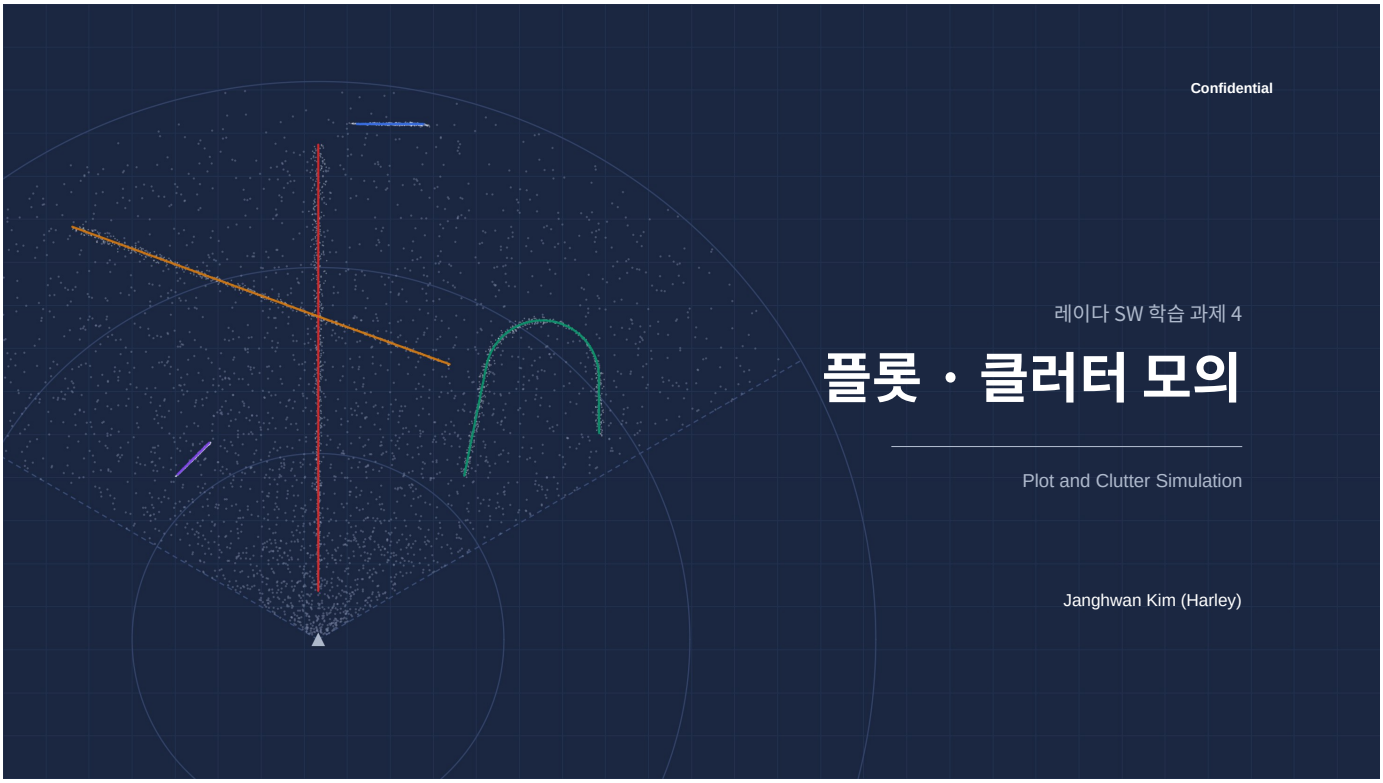
1 표지



발표 대본 표지 · 약 20초

안녕하십니까. 이번 과제의 주제는 플롯과 클러터 모의입니다.

지난 과제에서 만든 표적 궤적은 표적이 실제로 있던 자리, 즉 참값이었습니다. 이번에는 그 참값을 레이다가 잰 것 같은 측정값으로 바꾸고, 표적이 아닌 곳에서 생기는 가짜 탐지인 클러터도 함께 만들었습니다.

뒤에 나오는 결과 숫자와 그래프는 제가 만든 프로그램의 Core 를 그대로 돌려서 얻은 값입니다.

2 목차

INDEX

플롯 · 클러터 모의
PlotSim

Janghwan Kim (Harley)

01 플롯 3

02 클러터 11

03 구현 14

04 결과 20

2

발표 대본 목차 · 약 25초

순서는 네 부분입니다.

먼저 플롯이 무엇이고 어떻게 모의하는지, 이어서 클러터가 무엇이고 어떻게 모의하는지 개념을 설명드리겠습니다.

그다음 이 내용을 C 코드로 어떻게 구현했는지 보고, 마지막으로 난수 확인 결과와 표적 다섯 개의 모의 결과 화면을 보겠습니다.

각 부분을 시작할 때 그 부분에서 다룰 내용을 먼저 보여 드리겠습니다.

01. 플롯

플롯이 무엇인지 보고, 참값에 잡음을 더해 플롯을 모의하는 방법을 설명합니다.

1.1 플롯이란 4

1.2 플롯 생성 과정 5

1.3 플롯 모의 방법 6

1.4 측정 표준편차 p 7

1.5 GAUSS 정규난수 8

1.6 참값 x : 안테나 기준 거리 · 방위각 · 고각 9

1.7 잡음을 ECEF 가 아니라 안테나 기준 거리 · 각도에 더하는 이유 10

발표 대본 플롯 · 약 15초

먼저 플롯입니다. 이번 과제가 레이다 처리 흐름에서 어디에 있는지와 플롯이 무엇인지 보고, 참값에 잡음을 더해 플롯을 만드는 방법을 표준편차, 잡음, 참값 순서로 설명드리겠습니다. 끝으로 잡음을 ECEF 위치가 아니라 안테나 기준 거리와 각도에 더하는 이유를 보겠습니다.

4 1.1 플롯이란



발표 대본

플롯 · 약 80초

플롯은 레이다가 표적을 한 번 잰 결과입니다. 먼저 위쪽 흐름에서 이번 과제의 위치를 보겠습니다. 과제 3 에서는 맨 앞의 표적 궤적 모의로 표적이 실제로 있던 자리, 즉 참값을 만들었습니다. 이번 과제는 그다음 칸인 측정 모의이고, 여기서 만든 플롯과 클러터를 다음 과제의 추적 필터가 입력으로 받습니다.

아래 그림은 플롯이 만들어지는 과정입니다. 빔이 한 방향에 머무는 구간을 드웰이라고 합니다. 드웰 동안 받은 신호를 처리하면 거리와 각도로 나눈 칸마다 신호 세기가 나오고, 문턱을 넘은 칸을 히트라고 합니다. 첫 번째 그림처럼 표적 하나가 히트 여러 개를 내고, 왼쪽 위의 칸처럼 잡음이 문턱을 넘는 경우도 있습니다.

두 번째 그림처럼 붙어 있는 히트를 묶어 대표점 하나로 만든 것이 플롯입니다. 거리, 방위각, 고각과 잰 시각을 가집니다. 파란 원이 실제 표적 자리인데 플롯과 정확히 겹치지 않습니다. 측정에는 늘 오차가 있기 때문입니다.

세 번째 그림처럼 스캔마다 나온 플롯을 시간 순서로 이어 표적의 움직임을 추정한 것이 트랙이고, 트랙은 다음 과제에서 다룹니다.

5 1.2 플롯 생성 과정



발표 대본

플롯 · 약 75초

플롯을 만드는 과정은 두 가지입니다.

위쪽이 실제 레이다입니다. 표적에서 돌아온 신호와 수신기 잡음이 같이 들어오고, 이것을 IQ 데이터로 받습니다. 펄스 압축으로 긴 펄스를 좁게 모아 거리를 구분하고, 도플러 처리로 속도별로 나눠 움직이지 않는 클러터를 걸러 냅니다. CFAR 로 주변 잡음 크기에 맞춘 문턱을 정하고, 문턱을 넘은 칸을 히트로 뽑습니다. 한 표적에서 나온 히트를 하나로 묶고, 모노펄스 처리로 각도를 정밀하게 구하면 드웰 플롯이 나옵니다.

아래쪽이 이번 과제에서 하는 모의입니다. 표적 정보에 잡음을 더해 바로 드웰 플롯을 만들고, 가운데 단계는 모두 생략합니다.

생략해도 되는 이유는 추적기가 받는 것이 결국 플롯뿐이기 때문입니다. 신호처리가 플롯에 남기는 효과를 거리, 방위각, 고각 오차의 크기로 요약해서 넣으면 추적기 입장에서는 같은 성질의 입력이 됩니다. 그 오차의 크기를 어떻게 정하는지 이어서 보겠습니다.

01. 플롯

1.3 플롯 모의 방법

표적 · SNR · 레이다 파라미터로 거리 · 방위각 · 고각의 표준편차를 구하고, 그 크기의 가우시안 잡음을 성분마다 더합니다.

1 SNR 정하기

레이다 파라미터 (빔폭, 대역폭, 송신 전력 …) 로 표적 SNR 계산

실습: SNR = 20 고정

2 표준편차 p

$p_{rng} = c / (2B\sqrt{(2 \cdot SNR)})$

$p_{azi} = \theta_{az} / (K_M\sqrt{(2 \cdot SNR)})$

$p_{ele} = \theta_{el} / (K_M\sqrt{(2 \cdot SNR)})$

4.740 m · 0.4781° · 0.4939°

3 잡음 v

$v_{rng} \sim N(0, p_{rng}^2)$

$v_{azi} \sim N(0, p_{azi}^2)$

$v_{ele} \sim N(0, p_{ele}^2)$

성분마다 따로 뽑음

4 플롯 z

$z_{rng} = x_{rng} + v_{rng}$

$z_{azi} = x_{azi} + v_{azi}$

$z_{ele} = x_{ele} + v_{ele}$

참값에 잡음을 더함

예: 대공 표적 첫 스캔 (t = 0 s) 의 실제 값

성분	참값 x	잡음 v	플롯 z = x + v
거리	13,310.2362 m	+2.4453 m	**13,312.6815 m**
방위각	0.0000°	−0.3574°	**−0.3574°**
고각	1.2315°	−0.8396°	**0.3919°**

고각 −0.84° 이 플롯은 고각이 작게 나와 위도 · 경도 · 고도로 바꾸면 고도 300 m 가 아니라 105 m 에 찍힌다. 이런 오차가 스캔마다 새로 생긴다.

6

발표 대본

플롯 모의 · 약 70초

플롯을 모의하는 방법은 네 단계입니다.

첫째, SNR 을 정합니다. 원래는 빔폭, 대역폭, 송신 전력 같은 레이다 파라미터와 표적 거리로 SNR 을 계산하지만, 실습은 20 으로 고정했습니다.

둘째, SNR 과 레이다 파라미터로 거리, 방위각, 고각 오차의 표준편차 p 를 구합니다. SNR 20 이면 4.74 m, 0.478 도, 0.494 도입니다.

셋째, 평균이 0 이고 표준편차가 p 인 가우시안 분포에서 잡음 v 를 성분마다 따로 뽑습니다. 넷째, 참값 x 에 잡음 v 를 더한 것이 플롯 z 입니다.

아래 표가 실제 첫 스캔의 대공 표적입니다. 참값 거리에 2.4 m 정도를 더하고, 방위각과 고각에도 각각 잡음을 더했습니다. 이 플롯은 고각이 0.84 도 작게 나와서, 위도 · 경도 · 고도로 바꾸면 고도가 300 m 가 아니라 105 m 로 찍힙니다. 이런 오차가 스캔마다 새로 생깁니다.

7 1.4 측정 표준편차 p



발표 대본

플롯 모의 · 약 80초

표준편차 p 를 구하는 식을 자세히 보겠습니다. 세 식은 모두 분해능을 루트 2 SNR 로 나눈 꼴입니다. 분해능은 레이다가 두 표적을 따로 구분할 수 있는 최소 간격입니다.

거리 분해능은 광속 c 를 대역폭 B 의 두 배로 나눈 값입니다. 대역폭은 송신 신호가 차지하는 주파수 폭인데, 넓을수록 펄스를 좁게 모을 수 있어서 거리를 잘게 나눌 수 있습니다. 2 로 나누는 것은 전파가 표적까지 갔다가 돌아오기 때문입니다. 대역폭 5 MHz 를 넣으면 29.98 m 입니다.

각도 분해능은 빔폭을 모노펄스 기울기 상수 K_M 으로 나눈 값입니다. 빔폭은 안테나 빔이 퍼지는 각도로, 방위각은 좌우 폭 5.14 도, 고각은 위아래 폭 5.31 도입니다. 모노펄스 레이다는 합 신호와 차 신호의 비로 빔 안에서 표적 각도를 다시 재는데, 그 비가 각도에 따라 변하는 기울기가 K_M 이고 1.7 을 씁니다. 그래서 각도 분해능은 3.024 도와 3.124 도가 됩니다.

SNR 은 표적 신호가 잡음보다 몇 배 센지를 나타내고, 실습에서는 20 입니다. 신호가 셀수록 잡음에 덜 흔들리기 때문에 분해능보다 훨씬 작은 오차로 잴 수 있습니다. 루트 2 SNR 은 루트 40, 즉 6.325 이고, 세 분해능을 모두 이 값으로 나누면 거리 4.740 m, 방위각 0.4781 도, 고각 0.4939 도가 나옵니다.

8 1.5 GAUSS 정규난수

01. 플롯

1.5 GAUSS 정규난수

잡음은 참고 소스의 GAUSS 로 뽑습니다. GAUSS 는 균등난수 UNIRAN 12 개를 더해 정규분포를 만듭니다.

참고 소스 GAUSS(m, dev) NoiseGeneration.c

```
38  double GAUSS(double m, double dev)
39  {
40    int i;
41    double a = 0.;
43    for (i = 0; i < 12; i++)
44    {
45      a += UNIRAN();
46    }
48    return ((a - 6.0) * dev + m);
49  }
```

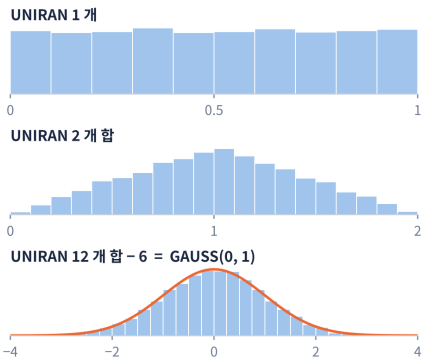
평균 0, 표준편차 1 이 되는 이유

UNIRAN 1 개	→	12 개 합	→	12 개 합 − 6
평균 1/2, 분산 1/12		평균 6, 분산 12 × 1/12 = 1		평균 0, 표준편차 1 → N(0, 1)

많이 더할수록 정규분포에 가까워진다 (중심극한정리)

플롯 계산: 성분마다 따로 호출 PlotSim.c

```
156  st_Out->st_Plot[nTarget].r   = st_Out->st_True[nTarget].r + GAUSS(0.0, st_Sigma.r);
157  st_Out->st_Plot[nTarget].az  = st_Out->st_True[nTarget].az + GAUSS(0.0, st_Sigma.az);
158  st_Out->st_Plot[nTarget].el  = st_Out->st_True[nTarget].el + GAUSS(0.0, st_Sigma.el);
```



주황 곡선: 표준정규분포 N(0, 1)
더하는 개수가 많아질수록 정규분포와 비슷해진다.

8

발표 대본

플롯 모의 · 약 80초

잡음은 참고 소스의 GAUSS 함수로 뽑습니다. GAUSS 는 0 과 1 사이의 균등난수 UNIRAN 을 12 개 더해서 정규분포를 만듭니다.

위 코드가 참고 소스의 GAUSS 입니다. UNIRAN 을 12 번 더한 값 a 에서 6 을 빼고, dev 를 곱한 뒤 m 을 더해 돌려줍니다.

평균 0, 표준편차 1 이 되는 이유는 아래 세 칸과 같습니다. UNIRAN 하나는 평균이 2 분의 1, 분산이 12 분의 1 입니다. 12 개를 더하면 평균은 6, 분산은 12 곱하기 12 분의 1 로 정확히 1 이 됩니다. 여기서 6 을 빼면 평균 0, 표준편차 1 인 표준정규분포 N(0, 1) 이 됩니다. 그래서 dev 를 곱하면 표준편차가 dev 가 되고, m 을 더하면 평균이 m 이 됩니다.

오른쪽 그림은 참고 소스의 UNIRAN 으로 실제로 뽑아 본 결과입니다. UNIRAN 하나는 평평하고, 두 개를 더하면 삼각형이 되고, 12 개를 더하고 6 을 빼면 주황색 표준정규분포 곡선과 거의 겹칩니다. 많이 더할수록 정규분포에 가까워진다는 중심극한정리입니다.

플롯을 계산할 때는 맨 아래 코드처럼 거리, 방위각, 고각마다 GAUSS 를 따로 부릅니다. 평균 자리에는 0, 표준편차 자리에는 앞에서 구한 성분별 p 를 넣고, 이렇게 뽑은 잡음을 참값에 더하면 플롯이 됩니다.

01. 플롯

1.6 참값 x : 안테나 기준 거리 · 방위각 · 고각

과제 3 결과는 ECEF 위치이므로, 레이다(플랫폼) 안테나 기준의 구좌표로 바꾼 뒤 잡음을 더합니다.

표적 ECEF — 과제 3 결과 (지구 중심 기준)

↓ `f_Trans_Ecef_To_Ned` (플랫폼 위치 · 위경도)

NED — 플랫폼 기준 북 · 동 · 아래

↓ `f_Trans_Ned_To_Body` (플랫폼 자세)

동체 — 플랫폼 선수 · 우현 · 아래

↓ `f_Trans_Body_To_Ant` (축 바꾸기)

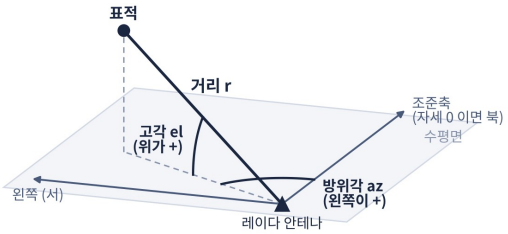
안테나 — 왼쪽 · 위 · 조준축 방향 직교좌표

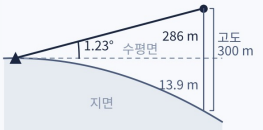
↓ `f_Trans_Ant_XYZ_To_Sph`

구좌표 x = (r, az, el) — 거리 · 방위각 · 고각

전시할 때는 거꾸로 구좌표 → 안테나 → 동체 → NED → ECEF → LLA (3.4)

참값 x = (r, az, el) 이 뜻하는 것





대공 표적, t = 0 s
정북 13.3 km, 고도 300 m
→ x = (13,310.2 m, 0°, 1.23°)
지면이 13.9 m 휘어 내려가
표적은 수평면 위 286 m
286 ÷ 13,310 → 1.23° (평면이면 1.29°)

9

발표 대본

플롯 모의 · 약 80초

잡음을 더하려면 먼저 참값을 레이다가 재는 형태로 바꿔야 합니다. 과제 3 의 결과는 지구 중심 기준의 ECEF 위치라서, 레이다 안테나 기준의 거리, 방위각, 고각으로 바꿉니다.

왼쪽 순서대로 플랫폼 위치를 기준으로 북 · 동 · 아래 좌표인 NED 로 바꾸고, 플랫폼 자세로 돌려 동체 좌표로, 축을 바꿔 안테나 좌표로 옮긴 다음 거리와 두 각도로 바꿉니다. 함수는 모두 제공된 좌표변환 코드를 그대로 썼습니다.

오른쪽 그림은 이렇게 구한 세 값이 무엇인지 보여 줍니다. 거리 r 은 안테나에서 표적까지의 직선거리입니다. 방위각 az 는 수평면에서 조준축으로부터 표적 쪽으로 돌아간 각도이고, 고각 el 은 수평면에서 표적을 올려다본 각도입니다. 방위각은 조준축 왼쪽이 플러스인데, 제공된 좌표변환 코드의 규약을 그대로 따른 것입니다. 플랫폼 자세가 0 이면 조준축이 북쪽이라 왼쪽은 서쪽입니다.

아래 상자는 대공 표적의 첫 위치입니다. 정북 13.3 km, 고도 300 m 이고, 바꾸면 거리 13,310.2 m, 방위각 0 도, 고각 1.23 도입니다. 300 을 13,310 으로 나누면 1.29 도인데 그보다 작게 나왔습니다. 상자 안 그림처럼 지구가 둥글어서, 13.3 km 앞의 지면은 레이다의 수평면보다 13.9 m 아래로 휘어 있습니다. 그래서 레이다가 보기에 표적은 수평면 위 286 m 에 있고, 286 을 13,310 으로 나누면 1.23 도가 됩니다. ECEF 에서 계산하니 이 효과가 자동으로 들어갔습니다.

전시할 때는 잡음을 더한 구좌표를 같은 길로 거꾸로 돌려 위도, 경도, 고도로 바꿉니다.



발표 대본

플롯 모의 · 약 100초

앞에서 참값을 안테나 기준 거리, 방위각, 고각으로 바꿨습니다. 과제 3 의 결과가 이미 ECEF 위치인데, 왜 거기에 바로 잡음을 더하지 않고 굳이 바꾼 다음에 더하는지 보겠습니다.

모의의 목표는 실제 레이다가 잴 때 생기는 오차를 그대로 흉내 내는 것입니다. 이 플롯으로 다음 과제의 추적 필터를 시험하기 때문에, 오차를 실제와 다르게 넣으면 엉뚱한 조건에서 시험하게 됩니다.

그런데 레이다는 표적의 X, Y, Z 좌표를 재지 않습니다. 왼쪽 그림처럼 전파를 보내고 돌아올 때까지 걸린 시간으로 거리를 잽니다. 13.3 km 앞 표적이면 88.8 마이크로초 뒤에 돌아옵니다. 방위각과 고각은 전파가 돌아온 방향으로 잽니다. 레이다가 직접 얻는 값은 안테나 기준 거리, 방위각, 고각입니다.

오차도 이 값에 생깁니다. 받은 신호에 잡음이 섞여서 도착 시각과 방향을 매번 조금씩 틀리게 잡기 때문입니다. 그 크기가 앞에서 구한 p, 거리 4.74 m, 방위각 0.478 도, 고각 0.494 도입니다. 그래서 같은 표적을 여러 번 재면 플롯이 그림의 빨간 범위처럼 거리 방향으로는 앞뒤로, 각도 방향으로는 옆으로 흩어져 찍힙니다. X, Y, Z 는 이 값으로 나중에 계산한 위치일 뿐입니다.

아래 두 줄이 이것을 정리한 것입니다. 실제 레이다는 거리와 각도를 재는 순간 오차가 섞이고, 그 값을 위치로 바꿔 전시합니다. 모의도 같은 순서로, 참값을 거리, 방위각, 고각으로 바꾸고, 실제 레이다에서 오차가 생기는 바로 이 자리에서 잡음을 더한 다음, 위도, 경도, 고도로 바꿉니다.

02. 클러터

클러터가 무엇인지 보고, 이번 과제에서 클러터를 어떻게 뽑는지 설명합니다.

2.1 클러터는 표적이 아닌 것에서 돌아온 탐지입니다 12
2.2 클러터 모의 방법 13

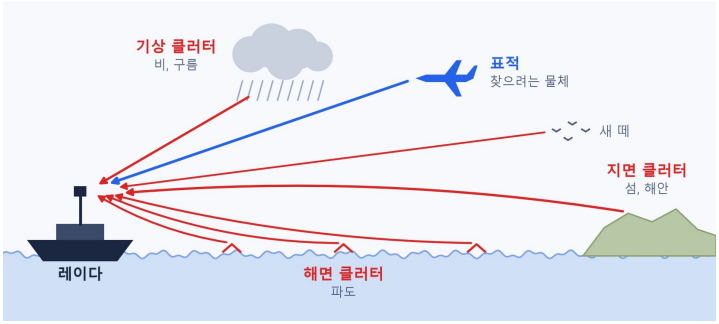
발표 대본 클러터 · 약 10초

다음은 클러터입니다. 클러터가 무엇인지 보고, 이번 과제에서 클러터를 어떻게 뽑는지 보겠습니다.

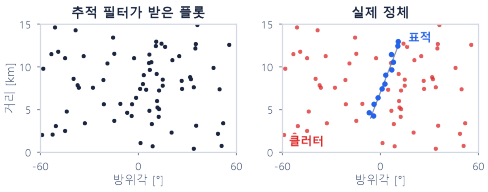
02. 클러터

2.1 클러터는 표적이 아닌 것에서 돌아온 탐지입니다

클러터가 생기는 곳



추적 필터가 받는 모습 (예시)



클러터도 거리 · 방위각 · 고각을 가진 플롯으로 들어온다.
플롯만 봐서는 표적을 구분할 수 없다.

실제 클러터를 그대로 만들기는 어렵다
바다 상태 · 지형 · 날씨까지 넣어야 하는 별도 연구 분야

그래서 단순하게 만든다
범위 안에 무작위로 생기는 클러터 → 클러터 속 추적 시험에 충분

12

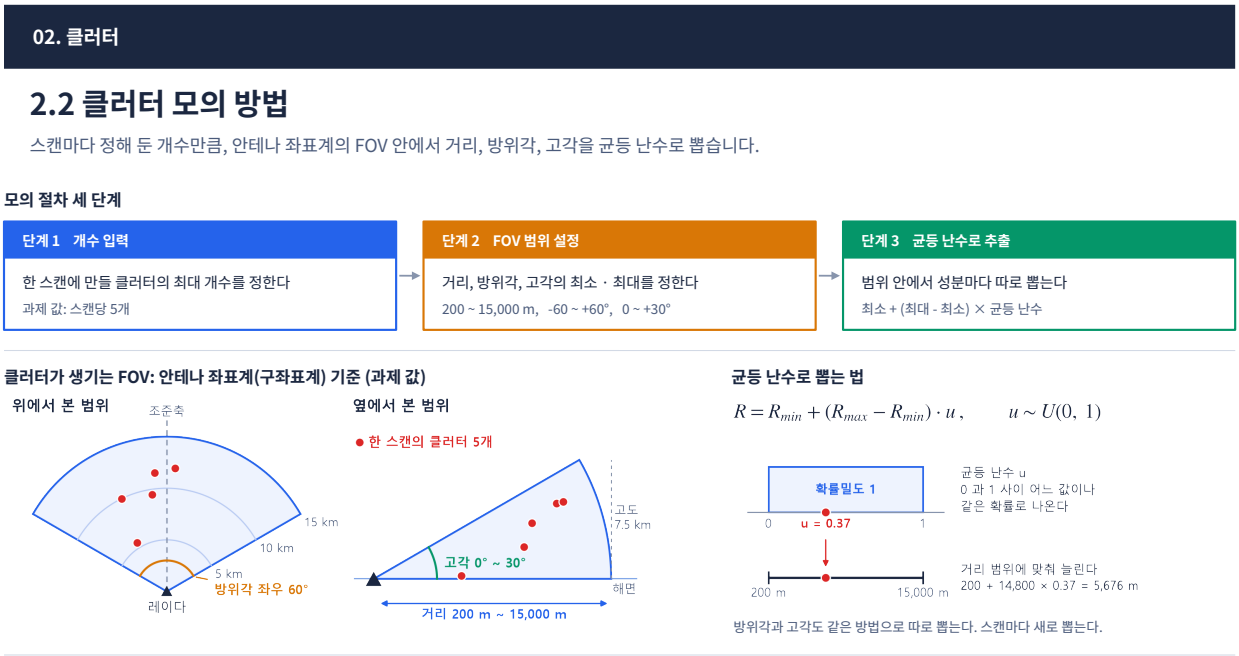
발표 대본

클러터 · 약 50초

레이다 전파는 표적에서만 돌아오지 않습니다. 왼쪽 그림처럼 파도, 섬, 비구름, 새 떼에서도 돌아옵니다. 이렇게 표적이 아닌 것에서 생긴 탐지가 클러터입니다.

클러터도 거리, 방위각, 고각을 가진 플롯으로 들어옵니다. 오른쪽은 추적 필터가 받는 모습의 예입니다. 왼쪽처럼 받은 플롯만 보면 모두 같은 점이라 어느 것이 표적인지 알 수 없고, 정체를 알고 나서야 오른쪽처럼 표적이 지나간 줄이 보입니다.

실제 클러터를 그대로 만드는 것은 바다 상태, 지형, 날씨까지 넣어야 하는 별도 연구 분야라서, 이번에는 범위 안에 무작위로 생기는 클러터로 단순하게 만듭니다. 클러터 속에서 표적을 잘 쫓는지 시험하기에는 이것으로 충분합니다.



발표 대본

클러터 · 약 70초

클러터를 만드는 방법은 세 단계입니다. 먼저 한 스캔에 클러터를 최대 몇 개까지 만들지 정합니다. 과제에서는 스캔당 다섯 개입니다. 그다음 클러터가 생길 수 있는 범위를 거리, 방위각, 고각별로 정합니다. 이 범위를 FOV 라고 하고, 값은 앞에서 본 안테나 좌표계 기준입니다.

아래 왼쪽 그림이 그 범위입니다. 거리는 200 m 에서 15 km 까지이고, 위에서 보면 조준축 좌우로 60 도씩 펼쳐진 부채꼴, 옆에서 보면 고각 0 도에서 30 도까지입니다.

마지막으로 이 범위 안에서 균등 난수로 값을 뽑습니다. 오른쪽 그림처럼 0 과 1 사이의 균등 난수를 뽑아서 범위에 맞게 늘려 주면 됩니다. 예를 들어 0.37 이 나왔다면 거리는 200 에 14,800 곱하기 0.37 을 더한 5,676 m 가 됩니다. 방위각과 고각도 같은 방법으로 따로 뽑고, 스캔이 바뀔 때마다 새로 뽑습니다.

거리와 각도 값을 고르게 뽑는 것이라서, 지도에 찍으면 레이다에 가까운 쪽이 더 촘촘하게 보입니다.

03. 구현

과제에서 요구한 순서대로 표준편차, 플롯, 클러터, 위도 · 경도 변환 코드를 봅니다.

3.1 표준편차 계산 (f_Pcs_CalcSigma) 15

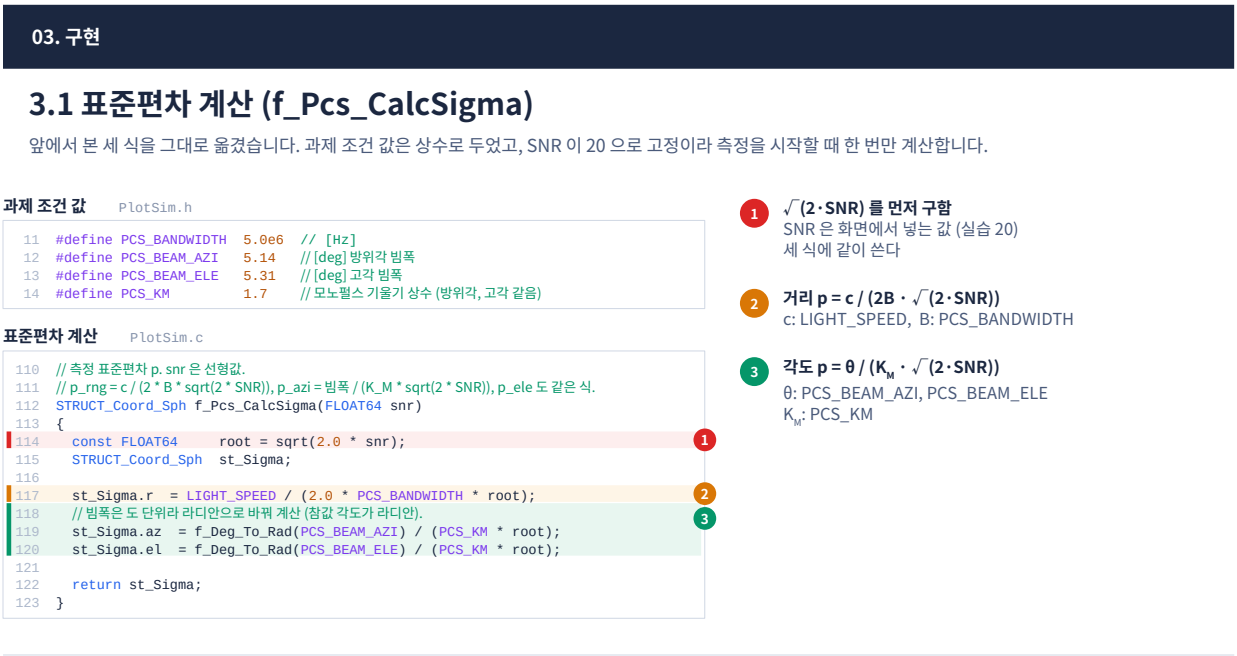
3.2 플롯 생성 (f_Pcs_Measure 앞부분) ..... 16 ~ 17

3.3 클러터 생성 (f_Pcs_Measure 뒷부분) 18

3.4 위도 · 경도 · 고도 변환 (f_Pcs_SphToLla) 19

발표 대본 구현 · 약 15초

이제 구현입니다. 과제에서 요구한 순서대로 표준편차를 구하는 코드, 플롯을 만드는 코드, 클러터를 만드는 코드를 보고, 마지막으로 지도에 찍기 위해 위도, 경도, 고도로 바꾸는 코드를 보겠습니다.



03. 구현

3.1 표준편차 계산 (f_Pcs_CalcSigma)

앞에서 본 세 식을 그대로 옮겼습니다. 과제 조건 값은 상수로 두었고, SNR 이 20 으로 고정이라 측정을 시작할 때 한 번만 계산합니다.

과제 조건 값 PlotSim.h

```
#define PCS_BANDWIDTH  5.0e6  // [Hz]
#define PCS_BEAM_AZI   5.14   // [deg] 방위각 빔폭
#define PCS_BEAM_ELE   5.31   // [deg] 고각 빔폭
#define PCS_KM         1.7    // 모노펄스 기울기 상수 (방위각, 고각 같음)
```

표준편차 계산 PlotSim.c

```
// 측정 표준편차 p. snr 은 선형값.
// p_rng = c / (2 * B * sqrt(2 * SNR)), p_azi = 빔폭 / (K_M * sqrt(2 * SNR)), p_ele 도 같은 식.
STRUCT_Coord_Sph f_Pcs_CalcSigma(FLOAT64 snr)
{
  const FLOAT64     root = sqrt(2.0 * snr);
  STRUCT_Coord_Sph  st_Sigma;

  st_Sigma.r  = LIGHT_SPEED / (2.0 * PCS_BANDWIDTH * root);
  // 빔폭은 도 단위라 라디안으로 바꿔 계산 (참값 각도가 라디안).
  st_Sigma.az  = f_Deg_To_Rad(PCS_BEAM_AZI) / (PCS_KM * root);
  st_Sigma.el  = f_Deg_To_Rad(PCS_BEAM_ELE) / (PCS_KM * root);

  return st_Sigma;
}
```

1 **√(2·SNR) 를 먼저 구함**
SNR 은 화면에서 넣는 값 (실습 20)
세 식에 같이 쓴다

2 **거리 p = c / (2B · √(2·SNR))**
c: LIGHT_SPEED, B: PCS_BANDWIDTH

3 **각도 p = θ / (K_M · √(2·SNR))**
θ: PCS_BEAM_AZI, PCS_BEAM_ELE
K_M: PCS_KM

15

발표 대본

구현 · 약 40초

표준편차를 계산하는 부분입니다. 위쪽은 과제에서 준 대역폭, 빔폭, 모노펄스 상수를 상수로 둔 것이고, 아래쪽이 그 값으로 표준편차를 구하는 함수입니다. 앞에서 본 세 식을 그대로 옮겼습니다.

먼저 세 식에 같이 들어가는 루트 2 SNR 을 한 번 구해 둡니다. 거리는 광속을 대역폭의 두 배와 이 값으로 나누고, 방위각과 고각은 빔폭을 K_M 과 이 값으로 나눕니다. 두 각도 식은 빔폭만 다르고, 빔폭을 라디안으로 바꿔 계산하는 이유는 코드 주석에 적어 두었습니다.

SNR 이 20 으로 고정이라 이 함수는 측정을 시작할 때 한 번만 부릅니다. 실제로 나온 값은 뒤의 결과 화면에서 보겠습니다.



발표 대본

구현 · 약 90초

플롯을 만드는 f_Pcs_Measure 함수입니다. 내용이 길어서 스캔 루프와 그 안에서 쓰는 변수 세 개부터 보겠습니다.

이 함수는 과제 3 의 결과인 st_Sample 과 스캔 수 600, SNR, 스캔당 클러터 수를 받습니다. 시작할 때 앞의 함수로 표준편차를 한 번 구해 둡니다.

st_Sample 은 과제 3 에서 0.1 초마다 남긴 기록을 차례로 담은 배열입니다. 칸 하나에는 그 시각과 표적 수, 플랫폼의 상태, 표적 다섯 개의 상태가 들어 있습니다. 스캔 루프는 nScan 을 0 부터 599 까지 늘리면서 이 칸을 하나씩 꺼내고, 칸 하나를 레이다가 한 번 잰 것으로 봅니다. 그래서 0.1 초 간격으로 600 번을 잽니다. 기록은 60 초까지 601 칸이지만 마지막 칸은 쓰지 않습니다.

오른쪽 그림은 nScan 이 1 일 때입니다. st_In 은 st_Sample 의 1 번 칸, 즉 0.1 초의 기록을 가리킵니다. st_Radar 는 그 기록 안의 플랫폼 상태를 가리킵니다. 레이다가 플랫폼에 실려 있으니 이 위치와 자세를 레이다의 위치와 방향으로 씁니다. 이번 시나리오는 플랫폼이 멈춰 있어서 매번 북위 32 도, 동경 126 도, 자세 0 도입니다.

st_Out 은 결과를 쓸 st_Scan 의 1 번 칸을 가리킵니다. 여기에 먼저 시각 0.1 초와 표적 수 5 를 옮겨 적고, 참값과 플롯, 클러터는 이어서 이 칸에 채웁니다.

03. 구현

3.2 플롯 생성 (f_Pcs_Measure 앞부분)

TargetSimCore/src/PlotSim.c

st_In 이 가리키는 기록의 표적마다 참값 x 를 구하고, 거리 · 방위각 · 고각에 GAUSS 잡음을 하나씩 더해 플롯 z 를 만듭니다.

```
141  for (nScan = 0; nScan < nScanNum; nScan++)
     ...
150    for (nTarget = 0; nTarget < st_In->nTargetNum; nTarget++)        ①
151    {
152      // 참값 x                                                      ②
153      st_Out->st_True[nTarget] =
           f_Pcs_EcefToSph(&st_In->st_Target[nTarget].st_PosEcef, st_Radar);
155      // 플롯 z = x + v, v ~ N(0, p^2)                               ③
156      st_Out->st_Plot[nTarget].r   = st_Out->st_True[nTarget].r + GAUSS(0.0, st_Sigma.r);
157      st_Out->st_Plot[nTarget].az  = st_Out->st_True[nTarget].az + GAUSS(0.0, st_Sigma.az);
158      st_Out->st_Plot[nTarget].el  = st_Out->st_True[nTarget].el + GAUSS(0.0, st_Sigma.el);
159      st_Out->st_PlotLla[nTarget]  = f_Pcs_SphToLla(&st_Out->st_Plot[nTarget], st_Radar);   ④
160    }
```

① **표적 루프**
st_In 의 표적 수 (5) 만큼, 표적마다

② **참값 x**
st_In 의 표적 ECEF 위치 → st_Radar 기준 거리 · 방위각 · 고각

③ **플롯 z = x + GAUSS(0, p)**
성분마다 잡음을 따로 하나씩

④ **지도용 위도 · 경도 · 고도**
바꿔서 저장 (3.4)

st_Out 이 가리키는 칸 st_Scan[1] 에 쓰는 값 nScan = 1 (0.1 s), 표적 2 (대공) 는 [1] 칸, 각도는 도로 바꿔 적음

st_True[1]	13,290.2 m, 0.0000°, 1.2335°	참값 x (②)
st_Plot[1]	13,286.7 m, −0.4007°, 0.8776°	플롯 z = x + 잡음 (③)
st_PlotLla[1]	32.1198°N, 126.0010°E, 217.4 m	지도용 (④)

17

발표 대본

구현 · 약 60초

이어서 스캔 루프 안쪽입니다. st_In 이 가리키는 기록의 표적 수만큼, 표적마다 같은 일을 합니다.

먼저 참값을 구합니다. st_In 에서 표적의 ECEF 위치를 꺼내, st_Radar 의 위치와 자세를 기준으로 안테나 기준의 거리, 방위각, 고각으로 바꾼 값입니다. 그다음 세 성분에 GAUSS 로 뽑은 잡음을 한 줄에 하나씩 더하면 플롯이 됩니다. 평균은 0 이고 표준편차는 성분별 p 입니다.

마지막 줄은 이 플롯을 지도에 찍을 위도, 경도, 고도로 바꿔 저장하는 부분이고, 뒤에서 따로 보겠습니다.

아래 상자는 0.1 초 스캔에서 대공 표적에 실제로 들어간 값입니다. 참값 거리 13,290 m 에 잡음이 더해져 플롯은 13,287 m, 방위각 −0.40 도, 고각 0.88 도가 되었고, 이 플롯을 위도, 경도, 고도로 바꾸면 고도 300 m 가 아니라 217 m 로 찍힙니다. 세 값 모두 st_Out 이 가리키는 st_Scan 의 1 번 칸에 들어갑니다.



03. 구현

3.3 클러터 생성 (f_Pcs_Measure 뒷부분)

같은 스캔에서 플롯을 만든 뒤, 정해 둔 범위 안에서 거리 · 방위각 · 고각을 균등 난수로 하나씩 뽑아 클러터를 만듭니다.

클러터 범위 (과제 조건) PlotSim.h

```
#define PCS_CLT_RNG_MIN  200.0    // [m]
#define PCS_CLT_RNG_MAX  15000.0
#define PCS_CLT_AZI_MIN  (-60.0)  // [deg]
#define PCS_CLT_AZI_MAX  60.0
#define PCS_CLT_ELE_MIN  0.0      // [deg]
#define PCS_CLT_ELE_MAX  30.0
```

클러터 생성 PlotSim.c

```
// 클러터: FOV 안 균등분포, 매 스캔 새로 뽑음
st_Out->nClutterNum = nClutterNum;                                  (1)
for (nClutter = 0; nClutter < nClutterNum; nClutter++)
{
  st_Clutter      = &st_Out->st_Clutter[nClutter];
  st_Clutter->r   = PCS_CLT_RNG_MIN +                               (2)
    ((PCS_CLT_RNG_MAX - PCS_CLT_RNG_MIN) * UNIRAN());
  st_Clutter->az  = f_Deg_To_Rad(PCS_CLT_AZI_MIN +
    ((PCS_CLT_AZI_MAX - PCS_CLT_AZI_MIN) * UNIRAN()));
  st_Clutter->el  = f_Deg_To_Rad(PCS_CLT_ELE_MIN +
    ((PCS_CLT_ELE_MAX - PCS_CLT_ELE_MIN) * UNIRAN()));
  st_Out->st_ClutterLla[nClutter] = f_Pcs_SphToLla(st_Clutter, st_Radar);   (3)
}
```

1. **스캔마다 5 개** — 화면에서 넣은 스캔당 클러터 수
2. **성분마다 UNIRAN() 하나** — 최소 + (최대 − 최소) × u, 각도는 도로 뽑아 라디안으로 바꾼다
3. **위도 · 경도 · 고도로 바꿔 저장** — 플롯과 같은 함수 (3.4)

18

발표 대본

구현 · 약 40초

같은 함수의 뒷부분, 클러터를 만드는 코드입니다. 위쪽은 과제에서 정한 클러터 범위를 상수로 둔 것입니다. 거리 200 m 에서 15 km, 방위각 −60 도에서 +60 도, 고각 0 도에서 30 도입니다.

스캔마다 표적 플롯을 다 만든 다음, 화면에서 넣은 개수만큼 클러터를 만듭니다. 과제대로 5 개를 넣었습니다.

클러터 하나마다 거리, 방위각, 고각을 UNIRAN 으로 따로 하나씩 뽑습니다. 식은 앞에서 본 최솟값 더하기 범위 폭 곱하기 균등 난수 그대로이고, 각도는 도 단위로 뽑은 다음 라디안으로 바꿉니다. 마지막 줄에서는 플롯과 같은 함수로 위도, 경도, 고도도 구해 둡니다.



발표 대본

구현 · 약 40초

플롯과 클러터는 레이다 기준의 거리와 각도라서 그대로는 지도에 찍을 수 없습니다. 과제에서 위도, 경도 위에 전시하라고 했으므로 f_Pcs_SphToLla 로 바꿉니다.

순서는 참값을 구할 때의 반대입니다. 구좌표를 안테나 직교좌표로 바꾸고, 축을 바꿔 동체 좌표로, 플랫폼 자세로 NED 로, 플랫폼 위치로 ECEF 로 옮긴 다음 마지막에 위도, 경도, 고도를 얻습니다. 코드의 번호와 오른쪽 그림의 번호가 같은 단계입니다.

왼쪽 아래에 적은 것처럼 자세와 위치는 그 스캔 시각의 플랫폼 값을 쓰고, 변환 함수 다섯 개는 모두 제공된 좌표변환 코드를 그대로 썼습니다.

04. 결과

난수가 이론 분포를 따르는지 검증 코드와 프로그램 화면으로 확인하고, 표적 5 개의 모의 결과 화면을 봅니다.

4.1 난수 확인 코드 (실습 2.1) 21
4.2 난수 확인 결과 (실습 2.1) 22
4.3 표적 5 개 모의 결과 (위도 · 경도 전시) 23

발표 대본 결과 · 약 15초

다음은 결과입니다. 실습 2.1 의 난수 확인은 이론으로 계산한 분포와 실제 코드가 만든 난수가 얼마나 맞는지 검증하는 것입니다. 검증 코드를 간단히 보고 프로그램 화면에서 결과를 확인한 뒤, 표적 다섯 개의 모의 결과 화면을 보겠습니다.

04. 결과

4.1 난수 확인 코드 (실습 2.1)

UNIRAN 과 GAUSS 가 이론 분포를 따르는지 검증하는 코드입니다. 10,000 개를 칸마다 세어 확률밀도로 바꾸고, 이론 확률밀도 · 평균 · 표준편차와 비교합니다.

검증 핵심 f_Pcs_FillHist PlotSim.c

```
26  for (nIndex = 0; nIndex < PCS_RAND_NUM; nIndex++)                    1
28    value  = (isGauss != 0) ? GAUSS(0.0, 1.0) : UNIRAN();
29    sum    = sum + value;
30    sumSq  = sumSq + (value * value);
33    nBinIndex = (INT32)floor((value - binMin) / st_Hist->binWidth);    2
35    if ((nBinIndex >= 0) && (nBinIndex < nBin))
37      st_Hist->nCount[nBinIndex]++;
52  for (nIndex = 0; nIndex < nBin; nIndex++)                            3
54    st_Hist->pdf[nIndex] = (FLOAT64)st_Hist->nCount[nIndex] /
        ((FLOAT64)PCS_RAND_NUM * st_Hist->binWidth);
59  st_Hist->mean  = sum / (FLOAT64)PCS_RAND_NUM;                        4
60  st_Hist->std   = sqrt((sumSq / (FLOAT64)PCS_RAND_NUM) -
      (st_Hist->mean * st_Hist->mean));
```

1. **10,000 개 뽑기**
   UNIRAN() 또는 GAUSS(0, 1)
   평균 · 표준편차를 구하려고 합과 제곱합도 더한다
2. **칸마다 세기**
   칸 번호 = (값 − 최솟값) ÷ 칸 폭 의 정수 부분
   UNIRAN 0 ~ 1 은 10 칸, GAUSS −4 ~ 4 는 32 칸
3. **확률밀도로 바꾸기**
   개수 ÷ (10,000 × 칸 폭) 이면 막대 넓이 합이 1
   이론 확률밀도와 같은 눈금에서 비교할 수 있다
4. **평균 · 표준편차**
   이론값 UNIRAN 0.5 · 0.2887, GAUSS 0 · 1 과 비교

SEED
SEED 를 되돌리지 않고 지난번에 멈춘 곳에서 이어 뽑아
창을 열 때마다 다른 10,000 개가 나온다 (f_Pcs_TestRand).
측정 모의는 SEED 를 따로 되돌려 써서 영향이 없다.

발표 대본

결과 · 약 60초

실습 2.1 의 난수 확인은 UNIRAN 과 GAUSS 가 이론 분포를 제대로 따르는지 검증하는 것입니다. 방법은 간단합니다.

먼저 난수를 10,000 개 뽑습니다. 평균과 표준편차를 구하려고 합과 제곱합도 같이 더해 둡니다. 뽑은 값은 칸 번호를 구해서 그 칸의 개수를 하나 올립니다. 칸 번호는 값에서 최솟값을 빼고 칸 폭으로 나눈 정수 부분입니다. UNIRAN 은 0 에서 1 까지 10 칸, GAUSS 는 −4 에서 4 까지 32 칸입니다.

다 세고 나면 개수를 10,000 곱하기 칸 폭으로 나눠 확률밀도로 바꿉니다. 이렇게 하면 막대 넓이를 모두 더한 값이 1 이 되어서 이론 확률밀도와 같은 눈금에서 바로 비교할 수 있습니다. UNIRAN 이면 모든 칸이 1 근처여야 하고, GAUSS 면 표준정규분포 곡선을 따라야 합니다. 평균과 표준편차도 UNIRAN 은 0.5 와 0.2887, GAUSS 는 0 과 1 이 나와야 합니다.

SEED 는 처음 값으로 되돌리지 않고 지난번에 멈춘 곳에서 이어서 뽑습니다. 그래서 창을 열 때마다 다른 10,000 개가 나옵니다. 측정 모의는 SEED 를 따로 되돌려 쓰기 때문에, 검증을 몇 번 하든 플롯 결과는 바뀌지 않습니다.

4.2 난수 확인 결과

04. 결과

4.2 난수 확인 결과 (실습 2.1)

프로그램을 켜고 처음 연 난수 확인 창입니다. 노란색은 첫 번째 값이 들어간 칸을 따라가며 계산한 것입니다.



22

발표 대본

결과 · 약 80초

프로그램을 켜고 처음 연 난수 확인 창입니다. 왼쪽이 UNIRAN, 오른쪽이 GAUSS 이고, 위의 막대가 코드로 구한 확률밀도, 주황 선이 이론 확률밀도입니다. 아래는 앞에서 본 네 단계를 실제 값으로 보여 주는데, 노란색이 첫 번째 값을 따라간 것입니다.

UNIRAN 의 첫 번째 값은 0.0966 입니다. 0.1 로 나누면 0.96 이라 정수 부분인 0 번 칸에 들어갑니다. 0 번 칸에는 10,000 개 중 999 개가 들어왔고, 이것을 10,000 곱하기 0.1 로 나누면 확률밀도가 0.999 로 이론값 1 과 거의 같습니다. 평균은 합을 10,000 으로 나눈 0.5026, 표준편차는 0.2892 로 이론값 0.5, 0.2887 과 거의 같습니다.

GAUSS 의 첫 번째 값 0.5953 은 4 를 더하고 0.25 로 나누면 18.38 이라 18 번 칸에 들어갑니다. 이 칸의 확률밀도는 0.3048 로 이론 0.3282 보다 조금 낮은데, 칸 하나에 800 개 안팎이라 이 정도는 흔들릴 수 있습니다. 바로 옆 17 번 칸은 이론보다 높아서, 전체로 보면 막대가 종 모양 곡선을 따라갑니다. 평균은 −0.0008, 표준편차는 0.9955 로 0 과 1 에 가깝습니다.

다시 뽑기를 누르면 다른 10,000 개를 뽑아 숫자가 조금씩 바뀌지만, 매번 이론값 근처에 모입니다. 그래서 두 난수가 의도한 분포를 따른다고 볼 수 있습니다.

04. 결과

4.3 표적 5 개 모의 결과 (위도 · 경도 전시)

프로그램을 실행하면 나오는 첫 화면의 지도입니다. 표적 5 개를 600 번 재서 만든 플롯 · 클러터를 위도 · 경도 위에 겹쳐 그립니다.

메인 창 지도 캡처를 넣을 자리 (tools/figures/ui_map.png)

1. **지도에 찍힌 것**
   선: 과제 3 참값 궤적 (표적 색)
   검은 점: 플롯, 회색 점: 클러터
   참값 궤적 표시를 끄면 플롯 · 클러터만 남는다

2. **플롯 오차 ≈ 이론 p**
   측정 모의 칸 요약 줄 (플롯 3000 개의 z − x)

표준편차	거리	방위각	고각
이론 p	4.740 m	0.4781°	0.4939°
플롯 오차	**4.800 m**	**0.4760°**	**0.4928°**

3. **개수**
   플롯 3000 개 = 600 스캔 × 표적 5 개
   클러터 3000 개 = 600 스캔 × 5 개

4. **클러터는 부채꼴 안에만**
   FOV (±60°, 15 km) 안에서 뽑음
   거리를 고르게 뽑아 레이다 가까이 촘촘

23

발표 대본

결과 · 약 80초

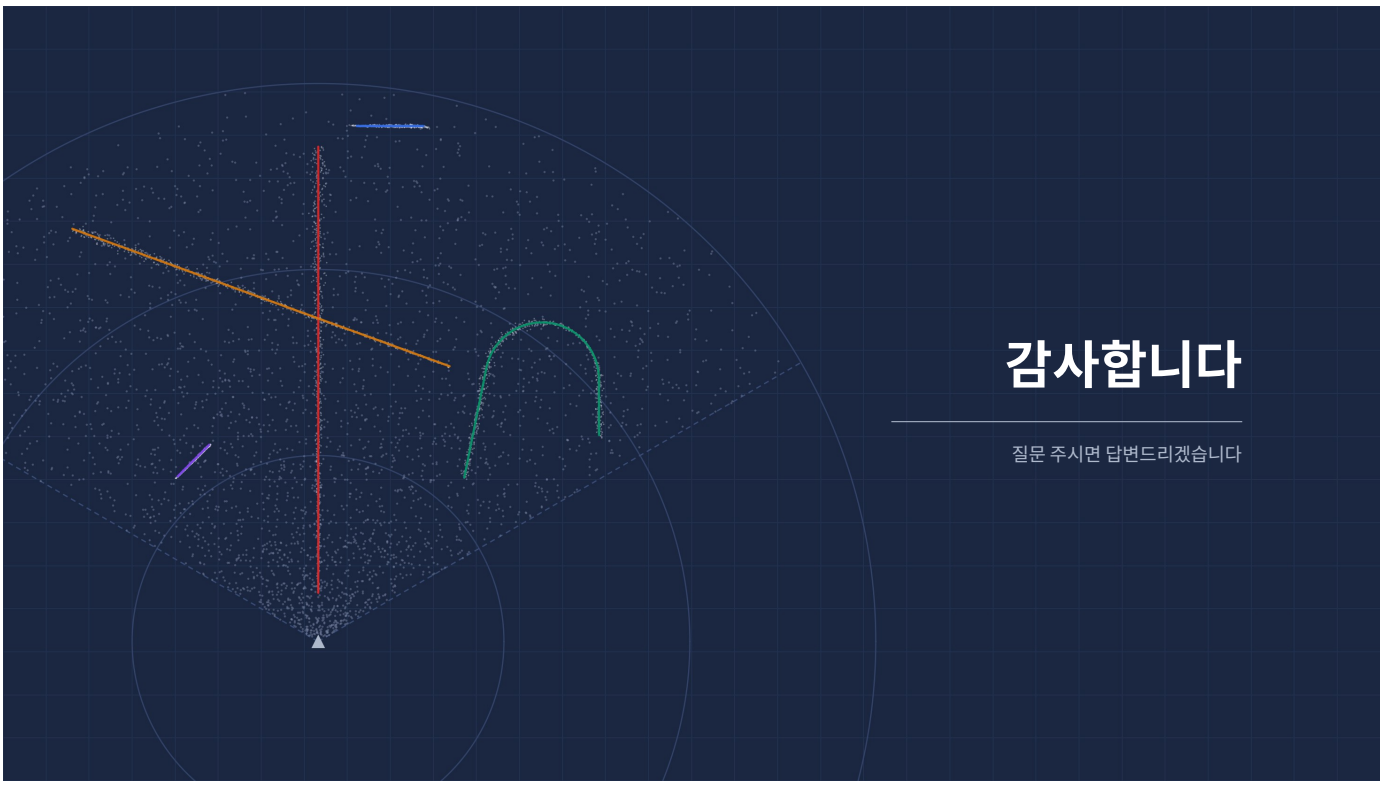
이 화면이 과제에서 요구한 결과입니다. 프로그램을 실행하면 바로 나오는 첫 화면의 지도로, 표적 다섯 개를 60 초 동안 0.1 초마다 잰 600 스캔을 위도와 경도 위에 모두 겹쳐 그립니다.

선은 과제 3 의 참값 궤적이고, 검은 점은 플롯, 회색 점은 클러터입니다. 플롯과 클러터는 레이다 기준의 거리, 방위각, 고각이라, 앞에서 본 변환 함수로 위도와 경도로 바꿔서 찍었습니다. 클러터는 방위각 ±60 도, 거리 15 km 의 부채꼴 안에서만 뽑았기 때문에 부채꼴 모양으로 찍히고, 거리를 고르게 뽑아서 레이다 가까이가 더 촘촘합니다. 참값 궤적 표시를 끄면 선은 사라지고 플롯과 클러터만 남는데, 다음 과제의 추적 필터가 받는 입력이 바로 이 점들입니다.

오른쪽 표는 메인 창 왼쪽 아래 측정 모의 칸에 나오는 요약 두 줄입니다. 위 줄은 SNR 20 으로 계산한 이론 표준편차 p 이고, 아래 줄은 플롯 3000 개에서 플롯과 참값의 차이를 모아 다시 구한 표준편차입니다. 거리 4.800 m, 방위각 0.4760 도, 고각 0.4928 도로 이론값과 거의 같습니다. 차이가 가장 큰 거리도 1.3 % 인데, 3000 개로 구한 표준편차라 이 정도는 흔들릴 수 있습니다. 넣은 잡음 크기대로 플롯이 흩어졌다는 것을 숫자로 확인한 셈입니다.

플롯 3000 개는 600 스캔에 표적 다섯 개, 클러터 3000 개는 600 스캔에 스캔당 다섯 개라서 개수도 맞습니다.

24 감사합니다



발표 대본 마무리 · 약 10초

이상으로 발표를 마치겠습니다. 감사합니다. 질문 주시면 답변드리겠습니다.